

scikit-learn pipelines

COMPLETE LEARNING GUIDE

What You'll Learn

Real feature preprocessing rarely stops at 'fit one model' — you typically need to scale numeric features, encode categorical ones, and apply the exact same transformations consistently to training data, test data, and any future new data the model sees in production. Doing this by hand invites a specific, common, and hard-to-detect bug called data leakage, where information from the test set improperly influences preprocessing decisions made using the training set. scikit-learn's Pipeline and ColumnTransformer classes exist specifically to prevent this by bundling preprocessing and modeling into a single object that's fit and applied consistently. This submodule covers why pipelines matter, how to build one that handles mixed numeric/categorical data, and how to combine it with cross-validation for robust, leakage-free evaluation.

1. The Problem Pipelines Solve: Data Leakage

A subtle but serious mistake is fitting a preprocessing step (like a scaler that standardizes numeric features, computing mean and standard deviation) on the entire dataset before splitting into train/test — because that scaler's mean and standard deviation are then computed partly from test data, information about the test set has 'leaked' into the training process, and your evaluation metrics will be optimistically biased in a way that won't hold up on genuinely new data. The correct approach is to fit every preprocessing step only on the training data, then apply (transform, not re-fit) that same fitted transformation to the test data — exactly the discipline a Pipeline enforces automatically.

2. The Pipeline Class: Chaining Steps Into One Object

sklearn.pipeline.Pipeline bundles a sequence of transformation steps and a final estimator into a single object that behaves like any other scikit-learn model — it has `.fit()`, `.predict()`, and `.score()` methods that run every step in order. Calling `.fit()` on a pipeline fits each preprocessing step and the model together in the correct order using only the data you pass in; calling `.predict()` on new data runs that same fitted sequence of transforms before making a prediction. This bundling is what makes it structurally impossible to accidentally apply a differently-fitted transform at prediction time than the one used at training time.

PYTHON

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

pipeline = Pipeline([
    ("scaler", StandardScaler()),
    ("classifier", LogisticRegression()),
])

pipeline.fit(X_train, y_train)          # scaler is fit ONLY on X_train
predictions = pipeline.predict(X_test) # X_test is transformed using X_train's
print(pipeline.score(X_test, y_test))
```

3. ColumnTransformer: Different Preprocessing for Different Columns

Real datasets mix numeric columns (which typically need scaling) and categorical columns (which typically need encoding into numbers, since most models can't consume text directly). ColumnTransformer applies a different transformer to different named subsets of columns and concatenates the results back into a single feature matrix — numeric columns go through a StandardScaler, categorical columns go through a OneHotEncoder (which converts each category into its own binary 0/1 column), and both pipelines run in parallel before being combined, all inside one object that plugs directly into an outer Pipeline as its first step.

PYTHON

```
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.pipeline import Pipeline
from sklearn.ensemble import RandomForestClassifier

numeric_features = ["age", "income"]
categorical_features = ["city", "membership_tier"]

preprocessor = ColumnTransformer([
    ("num", StandardScaler(), numeric_features),
    ("cat", OneHotEncoder(handle_unknown="ignore"), categorical_features),
])

full_pipeline = Pipeline([
    ("preprocess", preprocessor),
    ("model", RandomForestClassifier(random_state=42)),
])

full_pipeline.fit(X_train, y_train)
print(full_pipeline.score(X_test, y_test))
```

4. handle_unknown and Other Real-World Encoding Gotchas

OneHotEncoder(handle_unknown='ignore') is a small but important detail: without it, encountering a category at prediction time that never appeared in the training data (a new city, a new product code) raises an error and crashes the pipeline in production; with it, an unseen category is encoded as all-zeros instead, letting the pipeline degrade gracefully rather than fail outright. This is exactly the kind of production-readiness detail that's easy to skip in a notebook experiment but matters the moment a pipeline runs against live, evolving data.

5. Cross-Validating a Full Pipeline

Because a Pipeline behaves like any other scikit-learn estimator, it can be passed directly to cross_val_score or GridSearchCV, and critically, this correctly re-fits every preprocessing step from scratch on each fold's training portion — preserving the leakage-free discipline across the entire cross-validation process, not just a single train/test split. Running preprocessing outside the cross-validation loop instead (fitting a scaler once on the whole dataset, then cross-validating only the model) reintroduces the exact leakage problem pipelines exist to prevent — so always place your preprocessing inside the pipeline you cross-validate, never before it.

PYTHON

```
from sklearn.model_selection import cross_val_score

scores = cross_val_score(full_pipeline, X, y, cv=5, scoring="accuracy")
print(scores.mean())
# Each of the 5 folds independently fits its own scaler/encoder on that
# fold's training data – no leakage between folds.
```

6. Hyperparameter Tuning With GridSearchCV on a Pipeline

GridSearchCV systematically tries every combination of hyperparameter values you specify, using cross-validation to score each combination, and returns the best-performing one. Applied to a full pipeline, you can tune both preprocessing choices and model hyperparameters together in a single search, using the double-underscore syntax (step_name__parameter_name) to address a parameter that belongs to a specific step inside the pipeline.

PYTHON

```
from sklearn.model_selection import GridSearchCV

param_grid = {
    "model__n_estimators": [100, 200],
    "model__max_depth": [5, 10, None],
}

search = GridSearchCV(full_pipeline, param_grid, cv=5, scoring="f1")
search.fit(X_train, y_train)

print(search.best_params_)
print(search.best_score_)
```

Common Pitfalls

- Fitting a scaler or encoder on the full dataset before splitting into train/test, leaking test-set information into preprocessing.
- Manually re-implementing the same preprocessing steps for train and test data separately, risking subtle inconsistencies between the two.
- Omitting `handle_unknown='ignore'` on a `OneHotEncoder`, causing the pipeline to crash the first time it sees a category absent from training data.
- Running cross-validation on a model alone after preprocessing the whole dataset once, instead of cross-validating the full pipeline — this silently reintroduces leakage.
- Forgetting the double-underscore syntax (`step__param`) when specifying a `GridSearchCV` parameter grid for a pipeline, causing a confusing parameter-not-found error.
- Assuming `ColumnTransformer` preserves original column order in its output — the transformed feature matrix's column order follows the transformer list, not the input `DataFrame`.

Key Takeaways

- Pipelines exist primarily to prevent data leakage by guaranteeing preprocessing is fit only on training data and consistently applied elsewhere.
- `ColumnTransformer` lets you apply different preprocessing (scaling vs. encoding) to different columns within one unified pipeline step.
- `handle_unknown='ignore'` on `OneHotEncoder` is a small setting that prevents a real production failure mode.
- Always cross-validate the full pipeline, not the model alone after separately preprocessing the data — otherwise cross-validation itself leaks.
- `GridSearchCV` with `step__parameter` syntax lets you tune preprocessing and model hyperparameters together in one search.

Learn More

scikit-learn — Pipelines and Composite Estimators

<https://scikit-learn.org/stable/modules/compose.html>

scikit-learn — `sklearn.compose.ColumnTransformer` Reference

<https://scikit-learn.org/stable/modules/generated/sklearn.compose.ColumnTransformer.html>

scikit-learn — Tuning Hyperparameters (`GridSearchCV`)

https://scikit-learn.org/stable/modules/grid_search.html